going to be helpful. You can define a custom filter list by extending the `SimpleListFilter` class. This class has two methods: one that defines the values to show in the filter listing and another that determines the corresponding filter to run on the `QuerySet` used for the page.

Although I've been using some famous musicians as my examples, RiffMates is really about musicians seeking musicians. It is reasonable to assume anybody in the database actively looking for another musician is alive. Filtering by the last few decades is likely good enough.

A custom list filter class extends `admin.SimpleListFitler`, defines two attributes, and defines two methods. The attributes are `title` and `parameter_name`, specifying the title of the filter and the name of what is being filtered by. The methods are `.lookups()` and `.queryset()`, which are responsible for naming the filter choices and performing the filter query, respectively. Figure 5.8 shows a high-level version of this code.

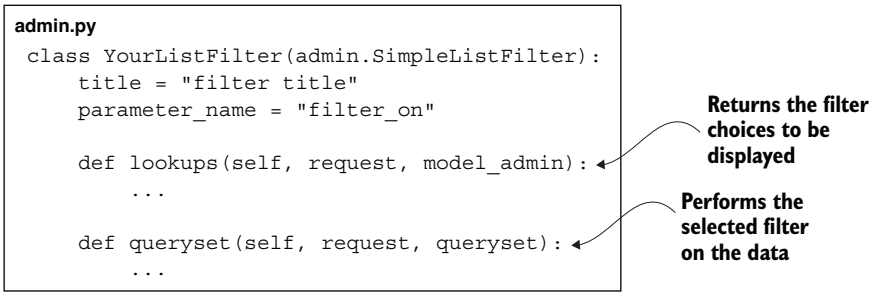


Figure 5.8 An overview of a custom list filter

The `.lookups()` method takes two arguments: the page's `request` object and a reference to the associated `admin.ModelAdmin` class. The method is responsible for returning an iterable of `tuple`s, each containing a short- and long-form display of the filter choice. The choices are strings, and the short form is used in the URL as a query parameter to the filtered page. The long form is what is displayed to the user as the filter link. This structure is similar to what Django expects in the `choices` attribute of a field constructor.

The `.queryset()` method is responsible for doing the actual filtering. It takes two arguments: the page's `request` object, and a `QuerySet`. When Django Admin calls this method, it has already preformed a query, which was used to construct the page. The resulting `QuerySet` object gets passed into `.queryset()`, allowing you to further perform actions on it. This method returns the modified `QuerySet` object with filtered contents.

Let's define a custom filter that provides the last 10 decades as date groups to filter by, calling it `DecadeListFilter`. Its `.lookups()` method returns an iterable of